Assessment 03 - Stream ciphers

Instructions
The assessment must be developed individually and submitted via Blackboard by **16/10/2026**. The submission should be a *zip* file containing all developed code and a short PDF report describing the code and discussing the results.

This assessment takes into account the use of Python and the Cryptography library. Installation instructions can be found here. However, you can choose a language and library of your preference.

Activities

Q1: For the items in *Q1*, take as a basis the `encrypt` function of the *One-Time Pad* (OTP) below.

```python
def encrypt(cleartext: str, keystream: bytes) -> bytes:
    text = cleartext.encode()
    result = bytes([x ^ y for x, y in zip(text,keystream)])
    return result
```

Q1.a: Write the corresponding `decrypt(ciphertext, keystream)` function.
Q1.b: One of the main challenges for the practical use of the OTP is the generation of unique random *keystreams* of the same length as the message whose confidentiality is to be protected. A developer decided to build a `pseudo-OTP` version that generates the *keystream* with Python's `random` module, using the current time as the seed (i.e., `random.seed(int(time.time()))` followed by `random.randbytes(n)`, available since Python 3.9). Implement this version and explain why the `random` module should not be used for cryptographic purposes. For further information, consult this reference.
Q1.c: The ciphertext below was produced with the `pseudo-OTP` version of item *Q1.b* on **15/09/2026** between **10:00** and **11:00** (UTC). Propose and implement a strategy to recover the cleartext. Afterwards, discuss why functions such as `os.urandom()` (see this reference) avoid this problem.

```
Ciphertext:
0xfed447773bd63bb2ebe347d620cae2056c40b47205189318412df6ddc1e15d384d4a159210
```

Q2: Assume that you have managed to intercept the traffic of a water treatment plant whose *Programmable Logic Controllers* (PLC) receive encrypted commands (i.e., OPEN, CLOSE, RESET, BYPASS, SHUTDOWN) using a *one-time pad scheme*. You know that the operator reuses a small set of randomly chosen *keystreams* to encrypt all the commands, but you do not know how many. Propose a strategy of cryptanalysis that allows discovering the plaintext of each command in the captured file, File-otp-Q2.txt, and the number of *keystreams* in use. Note that the ciphertexts in this file are represented in hexadecimal.

Q3: The list below consists of six ciphertexts produced using the same keystream of the OTP. Knowing that this cipher is vulnerable when used in this way (see more here), reveal the cleartext corresponding to *Ciphertext 3*.
Note that for this question, it is easiest to use a combination of automated analysis plus human insight and even occasional guessing. As long as you can decrypt them, it doesn't matter how you do it.
Obs. The cleartexts are in Portuguese, with removed special characters and spaces. Also, consider only upper-case text encoded with *UTF-8*.

```
Ciphertext 1:
0x98502a8cdda9ed53b972bde97a7c80f72495d57d044e11aed94d7e4a2e647fa67a8b57eb6ec7b032493f1f58989567106fbf5d1c7016
ad6a9b
Ciphertext 2:
0x9640268dddaded59ab72baea7a728cf0239ad87b135a16aec5427f5139666ab07d8d4de960d4bd21593f1342968b7c1e60b755126117
b87191adc2774476fcb4782b01787a633e78b4ff4134816aa5a43ed8
Ciphertext 3:
0x824e2e88c7adff59af72bdf0727380eb3790d07c17421baace4169462e766aa4619845
Ciphertext 4:
0x9640268dddaded59ab72baea7a728ffe3f84d06c174517aecb407b46316866af7a8552e06cc1b62750291f529c80681161a55f16770b
b06b9db2d5645767f0b17b2b07787363206da1fb4121856caca325d85ad92cc7cb51212b4cc58c
Ciphertext 5:
0x944f2e9ecba9fa54bc79a6eb757b8cfc399ac26c004416badf417550356266af7b9a41f769c1b127503f025f9e9f6a176fa55f00761e
```

```
b66b9ba6d3734571f2bd72231e7c7975316bb4f3
Ciphertext 6:
0x98502e9fcebdfc59ae73adf07e679dfe359bdf7017480aafc5416253306a7da2798b56e07ad0b13f56201b5598956d1e7db64b066711
ba6e95acd1715e73ebb5
```

Q4: After an audit, the water treatment plant from *Q2* replaced the *one-time pad scheme* with the stream cipher *ChaCha20*, using a secret key shared between the control centre and the PLCs and a unique nonce per message. You can use the implementation available in the Cryptography library (see more here). Note that, in this implementation, the 16-byte nonce includes the initial block counter.
From the public documentation of the PLCs, you know that the commands follow the fixed format `PLC=<id>;CMD=SET;NAOH_PPM=<value>;` , where `<id>` has two digits and `<value>` has five digits. You also know that the ciphertext below sets the sodium hydroxide concentration to `00100` ppm. Without knowing the key, produce a new ciphertext that, when decrypted by the PLC, sets the sodium hydroxide concentration to `11100` ppm. To allow you to check your attack, the key used by the plant is given below. However, your attack must not use it. Explain why the attack works, even though *ChaCha20* is considered a secure cipher.

```
Nonce:
0x00000000c351999e725e3f8d1d619bca
Ciphertext:
0xcaffea70ef5b03e9f898bebf13e125a8b81e1a75aea1f20e176bbbc0071a
Key:
0xe4f2510f07a2aee1a74aff224ab4cadf7cfab5977e489d077412ca49ee35d612
```

Q5: To simplify the firmware of the PLCs, the developers of the plant from *Q4* decided to hard-code the nonce used by *ChaCha20* (you can use the implementation available in the Cryptography library, see more here). You captured the two messages below, each transmitted together with its nonce. The first one is the periodic *heartbeat* that each PLC sends to the control centre, whose content is described in the public documentation and is always equal to `HEARTBEAT;PLC=07;STATUS=OK;NAOH_PPM=00100;PH=07.4;` . The second one is a command sent by the control centre to the same PLC. Propose a strategy of cryptanalysis that allows discovering the plaintext of the second message without knowing the key. What part of the second message could you recover if it were longer than the *heartbeat*?

```
Message 1 (heartbeat):
Nonce: 0x00000000a0acc5b086bb58000c5b0d31
Ciphertext:
0x58fd695cb8492a63b94400b0aed4aea89b7967106bbe1d8f9f5c4253b59ece1a2cc5629d17967fca02e28638fb6ffc083e1d
Message 2:
Nonce: 0x00000000a0acc5b086bb58000c5b0d31
Ciphertext: 0x40f46b33dc3c5461a03b6dafa8bdcec8e41163067bd61ac0e3760d3ca4bdb22b08b61d961c
```